

Selectie - din zona neordonată selectez minimumul și îl pun în zona ordonată.

selectie (L, [H/R]) :- min(L, M), delete(L, M, [H/R]),
selectie([H/R], R), write(R).

delete(L, [H/R]).

? selectie([3,7,2,5,4], Rez).

[2]

[7]

[5,7]

Alg Stabil $\Rightarrow$ păstrează relația de ordine parțială între elemente egale

$\hookrightarrow$ depinde de delete.

delete(H, [H/T], T) :- !.

delete(X, [H/T], [H/R]) :- delete(X, T, R).

} este stabil pt că
nu șterge primul
aparitiie, pt că faptul
este primul clouză.

min([H/T], M) :-

min(T, M),

M < H, !.

min([H/T], H).

$\hookrightarrow$

min-delete([H/T], M, [H/R]) :-

min-delete(T, M, R),

M < H, !.

min-delete([H/T], H, T).

} min + delete

$\hookrightarrow$

selectie se rescrie

$\hookrightarrow$
 $O(n^2)$

predicat stabil, deoarece din lista primul aparitiie
doar ero $\Leftarrow$ nu mai ero stabil

Insertare - adauga un element la locul potrivit in lista ordonata

$\text{in-ord}([H|T], E, [H|R]):$

$H < E, \text{in-ord}(T, E, R).$

$\text{in-ord}(\perp, E, [E|R]).$

Algor

$\text{insert}([H|T], E, [E, H|T]):-$
 $E < H, !.$

$\text{insert}([H|T], E, [H|R]):-$
 $\text{insert}(T, E, R).$

$\text{insert}(\perp, E, [E]).$

Sortarea prin insertie.

$\text{sort-insert}([H|T], R):-$

$\text{sort-insert}(T, Rp),$
 $\text{insert}(Rp, H, R).$

$\text{sort-insert}(\perp, \perp).$

- este stabil deoarece
nu se modifică ordinea
cu $E \leq H$

$\text{sort-insert2}([H|T], Acc, R):-$

$\text{insert}(H, Acc, Rp),$

$\text{sort-insert2}(T, Rp, R).$

$\text{sort-insert2}(\perp, R, R).$

$\text{sort-call}(\perp, R):- \text{sort-insert2}(\perp, \perp, R).$

- este stabil din cauza insertului

Interfața baze

swap([H₁, H₂ | T], [H₂ | R]) :- H₂ < H₁!,
swap([H₁, T], R).

swap([H | T], [H | R]) :-
swap(T, R).

swap([], []).

Algoritmul stabil
deoarece nu schimbăm
aceasta pune elem.
spate în ordine
corectă.

for(N, L, R) :-
N > 1!,
N₁ is N-1,
swap(L, L₁),
for(N₁, L₁, R).

for(_, R, R).

bubble sort.

while(J, 0) :-
swap(J, J_{int}),
J ≠ J_{int}!,
while(J_{int}, 0).

while(J, J).

altă variantă de
bubble sort.

swap2([H₁, H₂ | T], Flag, [H₂ | R]) :- H₂ < H₁!, Flag = 1,
swap2([H₁, T], Flag, R).

swap2([H | T], Flag, [H | R]) :-
swap2(T, Flag, R).

swap2([], _, []).

white (J, 0): -

swap2(J, Flog, Yut),
Non vaie (Flog), !,
white (Yut, 0).

white (J, J).

bubble sort cu noul
swap.

Flog = 1 Daca a fost relucubari
0 Daco nu - "

swap3([H1, H2 | T], [H2 | R], Flog): -
H2 < H1, !,

Flog = 1,

swap3([H1 | T], R, Flog).

swap3([H | T], [H | R], Flog): -

swap3(T, R, Flog).

swap3([J, J], 0): -

swap3([J, J], 1).

Daco te scriu nura mi se opreste la bucle infinita.

white (J, 0): -

swap3(J, Yut, F),

F = 1, !,

white (Yut, 0).

white (J, J).

Obs 1 bubble sort cu carda cu append !

① Se calculează înălțimea unui arbore binar

$height(n(l, r), H) := height(l, H_L), height(r, H_R),$
 $\max(H_L, H_R, H_P), H \text{ is } H_P + 1.$
 $height(n(-, out, r), H) := height(r, H_R), H \text{ is } H_R + 1.$
 $height(n(-, l, out), H) := height(l, H_L), H \text{ is } H_L + 1.$
 $height(n(-, nil, nil), 1).$

- parcurgere postordino
 (calcule după apel)
 în acest caz nu se poate
 folosi recursiv: max

sau cele trei clauze se înlocuiesc cu:

$height(nil, 0) \Rightarrow$ timpul de overhead cu stiva se dublează.

Teorema master: var(2)

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + 1$$

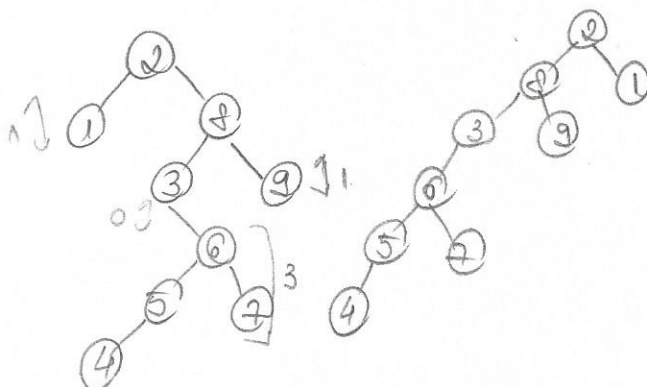
$\Downarrow$
 $O(n)$

$$T(n) = 3 \cdot T\left(\frac{n}{2}\right) + 1 > O(n)$$

$$T(n) = A \cdot T\left(\frac{n}{B}\right) + n^c$$

$\uparrow$
nr de apeluri recursive

② Arbore - binary: - numărul $\geq$ înălțime R iar proprietatea să se recurand.



$trouy(n(k, l, r), n(k, NR, NI)) := trouy(l, NI), trouy(r, NR),$
 $height(l, H_L), height(r, H_R),$
 $H_R > H_L, !.$

$trouy(n(k, l, r), n(k, NI, NR)) := trouy(l, NI), trouy(r, NR).$
 $trouy(nil, nil).$

Alt fel

$thouy(t(k, L, R), t(k, NR, NL))$: - $height(L, HL)$,
- $height(R, HR)$,
- $HR > HL$!,
- $thouy(L, NL)$,
- $thouy(R, NR)$.

PREORDINE

prelucram
parus data
clara

$thouy(t(k, L, R), t(k, NL, NR))$: - $thouy(L, NL)$,
- $thouy(R, NR)$.

$thouy(nul, nul)$.

Teorema master

$$T(n) = 2T\left(\frac{n}{2}\right) + n^1 \Rightarrow O(n \log n)$$

↓

Doi predicatul mai are un parametru care calc rezultato $\Rightarrow O(n)$

$thouy(t(k, L, R), t(k, PR, PL), H)$: - $thouy(L, PL, HL)$,
- $thouy(R, PR, HR)$,
- $HR > HL$!,
- H is $HR + 1$.

$thouy(t(k, L, R), t(k, PL, PR), H)$: - $thouy(L, PL, HL)$,
- $thouy(R, PR, HR)$,
- H is $HL + 1$.

Restul

$thouy(nul, nul, 0)$.

$$T(n) = 4T\left(\frac{n}{2}\right) + 1 \Rightarrow O(n^2) \text{ - timp de executie foarte mare}$$

↓

Solutie: inlocuim n cu un alt predicat.

pred-mok(A, B, C, L, R, L, R) :- A > B, !,
C is A+1.

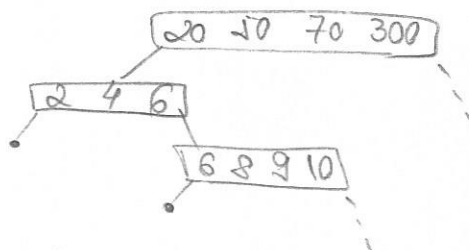
pred-mok(_, B, C, L, R, R, L) :- C is B+1.

trouff(T(K, L, R), T(K, NewL, NewR), H) :-
trouff(L, PL, HL),
trouff(R, PR, HR),
pred-mok(HL, HR, H, PL, PR, NewL, NewR).
trouff(nil, nil, o).

$T(n) = 2T(\frac{n}{2}) + 1 \Rightarrow O(n)$ - limita teoretică.

③ Arbore binar de căutare ale cărui noduri conțin liste ordonate.
Se cere:

- să se găsească/contine după primul și doare primul aparținere a unui element atomic din struct. originală.
- folosind append să se trouff structura inițială într-o listă plată ordonată
- folosind liste diferite - " -
- să se repete a), b), c) dacă listele din noduri sunt incomplete
- " - dacă arborele este incomplet.



a) search(T(List, L, R), H) :- member(H, List), !.
search(T([H|_], L, _), X) :- X < H, !, search(L, X).
search(T(_, _, R), X) :- search(R, X).
search(nil, X) :- !, fail.

Heff:

$\text{search}(H(k, L, R), S) :- \text{head}(k, HK), S < HK, !, \text{search}(L, S).$

$\text{search}(H(k, L, R), S) :- \text{member}(S, k), !.$

$\text{search}(H(k, L, R), S) :- \text{last}(k, LK), S > LK, !, \text{search}(R, S).$

$\text{head}([H1_], H).$

$\text{last}([L_IT], L) :- \text{last}(T, L).$
 $\text{last}([H], H)$ } more efficient in ordered accosta

b)

$\text{list-ord}(H(k, L, R), List) :- \text{list-ord}(L, ListL),$
 $\text{list-ord}(R, ListR),$
 $\text{append}(k, ListL, LA), \text{append}(ListR, LA, List)$

$\text{list-ord}(H(k, L, out), List) :- \text{list-ord}(L, ListL),$
 $\text{append}(ListL, k, List).$
 $\text{list-ord}(H(k, out, R), List) :- \text{list-ord}(R, ListR),$
 $\text{append}(k, ListR, List).$ } not opst.

$\text{list-ord}(out, []).$

$\text{thous}(k, KH, KT),$

c) $\text{list-ord}(H(k, L, R), H1, TR) :- \text{list-ord}(L, H1, KH),$
 $\text{list-ord}(R, KT, TR),$

$\text{list-ord}(out, H, H).$

$\text{thous}([HIT], [HIF], L) :- \text{thous}(T, F, L).$

$\text{thous}([], L, L).$

d) a) - se sclerubă : cucurbit
lact.

b) - se sclerubă prin intermediul append.

c) - se sclerubă trough

e) a) - condiție de oprire - prino.

b) - " -

c) - " -

① BST $\xrightarrow[\text{recursiv}]{\text{linear}}$ Ord. List

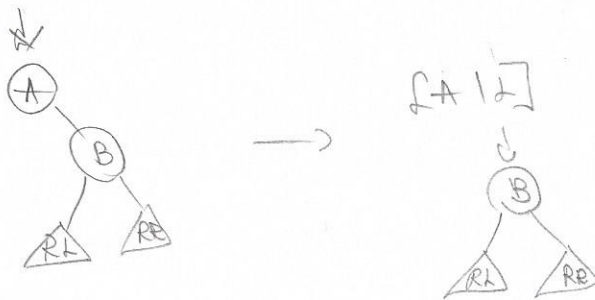
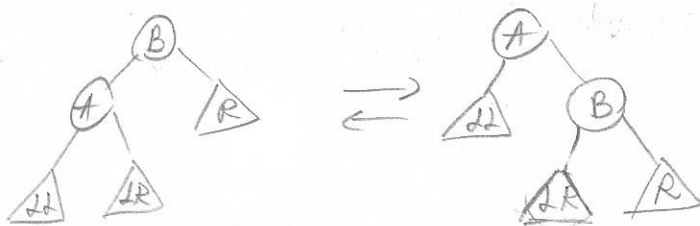
Recursivitate liniară: pe $\neq$ totuși merge doar un apel recursiv.

! $\neq$ Recv de cod cu recursivitate liniară se poate transp. în cod. iterativ

Avantaj pt recursivitate $\Rightarrow$ timp $\neq$ memorie mai mare

! Dacă vrem să reținem de recursivitate multiple

- for
- crearea propriilor stive



transp ($t(\text{key}, L, R)$, $[key / list]$): - $\text{more}(t(\text{key}, L, R), RP)$,
 $\text{transp}(RP, list)$.

$\text{transp}(out, list)$.

$\text{more}(t(\text{key}, t(K, L, R), R), t(K, L, t(\text{key}, LR, R)))$,
 $\text{more}(out, out)$.

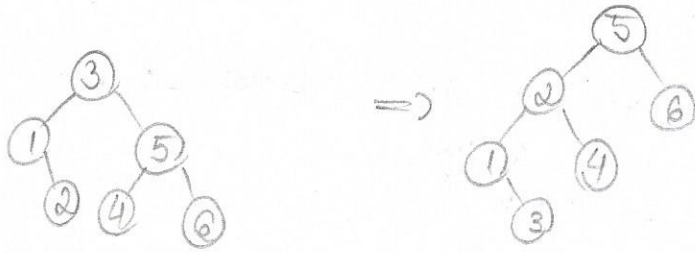
Alfel:

$\text{linear}(t(t(LL, A, LR), B, R), list)$: -
 $\text{linear}(t(LL, A, t(LR, B, R)), list)$.

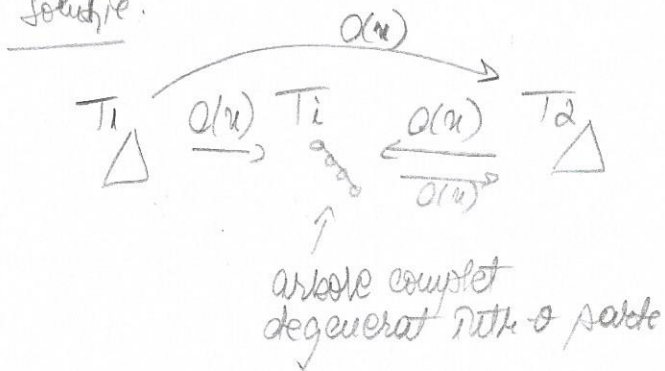
$\text{linear}(t(out, A, T), [A / list])$: -
 $\text{linear}(T, list)$.

$O(n)$

② Dorechki / Am cō + arb. binar de cōtare cu n key poate fi
 thourf. n + alt arb de cōtare cu aceleasi n key n timp
 biaz.



Solutie.



Cum putem thourf efectiv.

$T_1 \xrightarrow{O(n)}$ listă diferentă.

$T_2 \rightarrow T_3$ (arb cu aceleasi formă, dar n elci arbor -)
 e parcureru si copreu elcite din listă.

③ Calculati diametrul unui arbore binar.

Diametrul unui graf $\Rightarrow$ cea mai mare dist minimă ntre 2 noduri
 din graf.

diametru (L, R, H) : - diametru $(L, \Delta L, H_L)$,
 diametru $(R, \Delta R, H_R)$,
 $\max(H_L, H_R, H)$, H is $H+1$,
 H is $H_1 + H_2$,
 ④ $\max(\Delta L, \Delta R, H, \Delta)$.

diametru $(nil, -1, 0)$.

Deci apelam aici height $\Rightarrow O(\log n)$

GRATURI

④

Se dă un graf augmentat arând asociate culorilor nodurilor Aofe.
Să se genereze subgraful arând doar nodurile (și arcele aferente)
cu culoare distinctă de culoarea tuturor vecinilor.

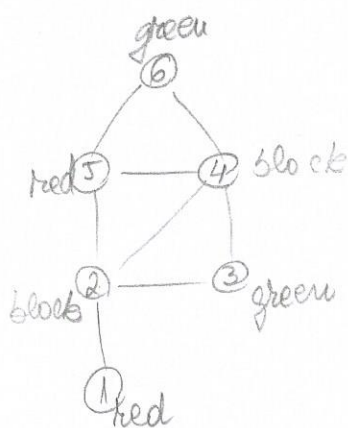
struct augmentată: pe lângă rf tipică se adaugă rf. carnot pb
în cazul nostru: culoarea

Subgraf.

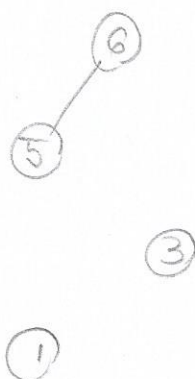
$$G(V, E) \Rightarrow G' = (V', E')$$

$$V' \subseteq V$$

$$E' \subseteq E$$



$\Rightarrow$



edge(x, y, c).
neighb(x, [], c).
y

! Could resolve pb constr subgraful, nu stăpene în graful inițial.

is-good(x) :- edge(x, y, c),
edge(y, z, c), !, fail.

is-good(-).

generate :- is-good(x),

is-good(y),
assert(edge(x, y, c)),
generate.

generate

Altfel:

neighb(1, [2])
neighb(2, [1, 3, 4, 5]).

col(1, red),
col(2, block).

build - nodes: -

- neighs(x, t),
- col(x, c),
- good(c, t)
- assert($col_s(x, c)$),
- fail.

build - nodes.

good($c, \lfloor H/2 \rfloor$): -

- col(H, c), !, for

good($c, \lfloor -1/2 \rfloor$): -

- good(c, t).

good($c, \lfloor 2 \rfloor$).

predicat de verificare

build - edges: -

- col - s($x, -$),
- neighs(x, t),
- new - neighs($t, t+1$),
- assert($neighs - s(x, t+1)$),
- fail.

build - edges.

new - neighs($\lfloor H/2 \rfloor, \lfloor H/2 \rfloor$): -

- col - s($H, -$), !,
- new - neighs(T, R).

este nod al subgrafului

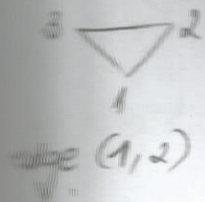
new - neighs($\lfloor -1/2 \rfloor, R$): -

- new - neighs(T, R).

new - neighs($\lfloor 2 \rfloor, \lfloor 2 \rfloor$).

predicat de construire

subgraf: - build - nodes, build - edges.



neigh (1, 2, 3).

! linutare: - reprezentare specială pt un nod izolat

edge (4, ...)

neigh (4, 1)

factu -

21 - edge (k, y): -

edge (k, y) ;
edge (y, k).

Acet predicat se adăpă pt
primu leproz. de enol de
care grafu este necerentat.

edge (k, y) $\equiv$ { neigh (k, L),
members (y, L) }.

- apel nedeterminat

check (k): -

is-edge (y, z),
not (verify (k, y)) !,
fail

could dorin să
verificu cea

check (k): - ...

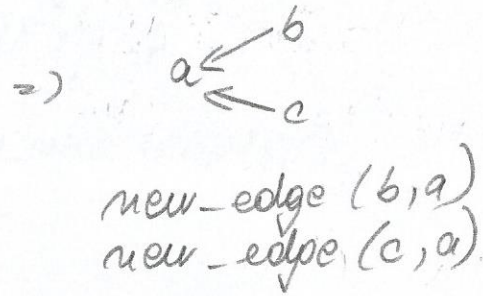
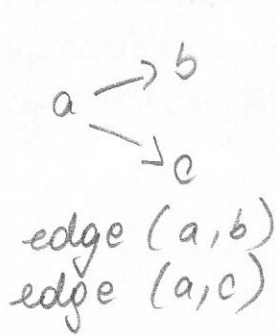
do (k): - ;

is-edge (y, z)
solve (k, y)
fail

could mean să cruth
cea escul nu este
asociat cu !

do (k): -

① Se dă un graf direcționat reprezentat prin arce. Să se constr. graful având aceeași mulțime de noduri și arce, doar orientarea arcelor în sens opus.



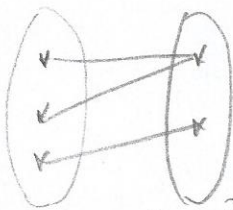
construct $\therefore$ - $\text{edge}(x, y)$,
 $\text{assert}(\text{new_edge}(y, x))$,
 fail.

construct. } nodurile izolate sunt puse în ordine numerică.

② Avându-se un graf conex neorientat, să se det. dacă graful este bipartit. (mult. vârfurilor poate fi partitionată a. i. arcele să fie între noduri care aparțin part. tlor disjuncte)

$$V = V_1 \cup V_2, \quad V_1 \cap V_2 = \emptyset \quad \wedge \nexists (x, y)$$

$$\left\{ \begin{array}{l} \nexists x \in V_1 \wedge y \in V_2 \\ \text{sau} \\ x \in V_2 \wedge y \in V_1 \end{array} \right.$$



- ciclu $\Rightarrow$ nu este bipartit

$\nexists x \in V_1 \nexists (x, y) \in E$

(1)	$y \in V_2$	✓
(2)	$y \in V_1$	$\rightarrow$ nu este bipartit
(3)	$y \Rightarrow V_2$	

~~$\text{part}([x/V_1], R_L) \therefore - \text{neighb}(x, [H/T])$
 $\text{part}([H/V_1], R_L).$~~

bipartit: - $\rightarrow$ set (x, v) ,

$\rightarrow$ edge (x, y)

$\rightarrow$ not (verify (y, v)), !, fail.

output.

bipartit: - findall $(x, \text{set}(x, v_1), v_1)$,
findall $(y, \text{set}(y, v_2), v_2)$ $\rightarrow$ partitite.

(2) verify (y, v) : - set (y, v) , !, fail.

acum se pot extrage ciclurile de lungime nupara.

(1) verify (y, v) : - set $(y, -)$.

(3) verify (y, v) : - opus (v, v_1) ,
asserta (set (y, v_1)).

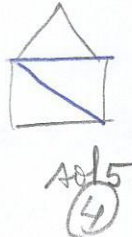
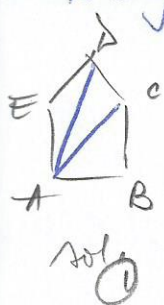
opus (v_2, v_1) .

opus (v_1, v_2) .

bip: - edge (x, y) ,
asserta (set (x, v_1)),
asserta (set (y, v_2)),
bipartit.

Teore - Sa extragem ciclurile de lungime nupara.

(3) Se da un graf de carti nuchii reprezentata un poligon convex.
Sa se realizeze toate descompunerile posibile ale poligonului
in triunghiuri adiacente.



Obs: Putem sa presupunem ca graful este dat sub forma
 $L = [a, b, c, d, e]$

construct (L, y) : - member (y, L) , member (x, L) ,
not (edge (y, x)),
asserta (edge (y, x)),

Altfel

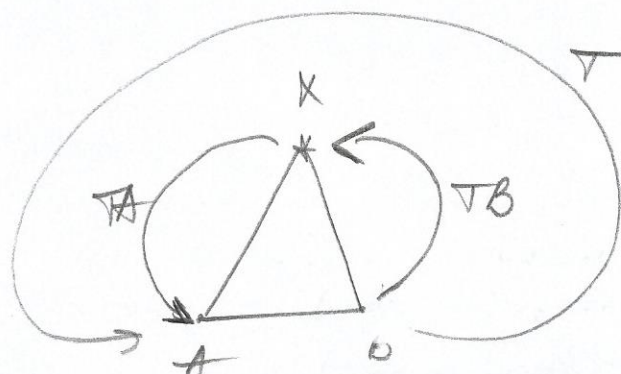
$\text{dec}([A, B], \perp, \perp).$

$\text{dec}([A, B | T], [t(A, B, X) | \neg], \perp): -$

$\text{append}(TB, [X | TA], T),$

$\text{dec}([X, B | TB], \neg, T),$

$\text{dec}([A, X | TA], T, \perp).$



Soluție obținută

? - $\text{dec}([a, b, c, d, e], R, \perp).$

$\Leftarrow R = [t(a, b, c), t(a, c, d), t(a, d, e)];$ ①

Teză: se abordează repetitivă notă a grafului de 2.